lux Connection-Lease Reaping: a Z Specification

Formal model of the Hub client registry's lease sweep and its coupling to scene ownership – refined to

September 2026

1 Introduction

Round 1 of this model proved a single property: once a connection's lease lapses, one atomic `Reap` both drops it from the registry and releases every scene it owned. That property type-checked, model-checked clean against all four of its invariants, and the implementation built from it passed 4,730 tests and an independent re-verification. It was also too narrow to be true of the code it claimed to model. The shipped system does not have one departure operation; it has three, and the model's single `Reap` silently assumed all three were the same thing.

A four-way review of the implementation found three separate places where a connection can leave the registry without its ownership being released – three holes the single-`Reap` abstraction hid because it never represented more than one way to leave:

H1 — destructive-read-sweep. `HubClientRegistry .named_sessions` (and, through it, `live_sessions` and `repos`) calls `_sweep_locked` on every read, which drops a lapsed session from the registry's own dict and returns what it dropped – but six call sites read through `named_sessions` /`live_sessions` without ever forwarding that return value to `OwnerTracker`. A read that happens to run after a connection's lease has lapsed silently deregisters it, ownership untouched. The one call site that *does* forward the return value (`HubDisplay._reap_and_release`, via `HubClientRegistry .reap`) is a write path, not a read.

H2 — graceful-departure-without-release. `HubDisplay .drop_connection` – the ordinary path an MCP session's clean disconnect or the SDK's own idle-timeout takes, via `session_cleanup.py`'s teardown leg and `lifecycle.disconnect_connection` – calls `HubClientRegistry.discard` unconditionally and never touches `OwnerTracker` at all. Round 1's fidelity control (`connection_lease_reaping_no_r` already modeled "no release on departure," but it modeled that as a variant of the *lease-lapse* path. It is not: it is this path, the one every ordinary session end takes, lapsed or not.

H3 — self-reap-without-renew. `HubDisplay.apply` calls `self._reap_and_release()` – an unconditional sweep of *every* lapsed session, itself included – before it does anything else, and never calls anything that renews the calling connection's own lease. A connection that only ever contacts the Hub to patch an already-shown scene never renews outside that first `show`; once its lease's length has passed, its own next write sweeps and releases itself before performing the very write that should have kept it alive.

All three share one root: the code has no single place where "leave the registry" and "release what you owned" are the same atomic step for *every* trigger. The operator's ruling fixes that with three decisions this model encodes and does not re-open:

1. **One coordinator.** The only way a connection leaves the registry is one atomic deregister-and-release step. Reads never remove anyone. Every departure trigger – graceful disconnect, the SDK's idle-reap, and a lease lapsing – funnels through that one step.

2. **Release ownership, keep content.** Departure releases a connection's scenes to `unowned`; it does not tear the scenes down. A reconnecting client reclaims through the ordinary unowned-claim path – I4, unchanged.

3. **Renew on contact, including a write.** Any authenticated contact renews the caller's own lease, and that now explicitly includes the ownership-gated write path (`apply`), which round 1 left uncovered.

The first cut of this round realized "a lease lapsing" entirely as a *side effect* of some other party's activity – `Depart`'s external signal, or `Claim`'s embedded sweep of others. The leader's ruling on that cut's own surfaced gap adds a fourth decision:

4. **The lease is a real backstop, not just a bookkeeping field.** A lapsed lease must eventually discharge on its own, with no read, no other connection's write, and no detected disconnect required – exactly what "the 30-second lease exists for this" promises, and exactly what a purely pull-based design cannot deliver for a killed process on an otherwise idle Hub.

Eight operations realize this: `Connect` and `Renew` are unchanged from round 1. `TransportDies` is unchanged. `LeaseLapse` drops the transport-death precondition it carried in round 1 – H3 is exactly the case of a lease lapsing under a transport that never dies, and round 1's guard could not represent that case at all. `Read` is new: it is the non-destructive query round 1 never modeled, because round 1 never needed to distinguish a read from anything else. `Depart` is the single coordinator – one schema, both for a graceful disconnect and for the SDK's idle-reap, since both concretely call the same `drop_connection` today and neither depends on lease state. `Claim` models `apply`: it renews the caller's own lease first, then sweeps every *other* lapsed connection and releases what each owned, then performs the ownership-gated write against the result – one atomic step, matching `apply`'s own docstring, "reaps and releases first, before the ownership check." `TimedReap` is this addendum's addition: the identical atomic deregister-and-release predicate `Depart` and `Claim`'s sweep already perform, guarded only on `lease c? = llapsed` – no other connection, no read, and no external signal is a precondition of it firing.

Four properties survive from round 1, one strengthened:

I1 — transport-gone implies *unconditionally* eventually reaped. Strengthened by this addendum. A connection whose transport has died cannot renew, and nothing can indefinitely block `TimedReap` once its lease lapses – not even the presence of every *other* connection idle, every read absent, and no disconnect ever detected. Round 1's version of this property read "nothing can indefinitely block `Depart` once it applies," which is true but was silently relying on *something* eventually triggering `Depart` or a peer's `Claim` – exactly the dependency the bead's own evidence (a connection persisting 25.79 hours with nobody else ever writing or disconnecting it) shows cannot be assumed.

I2 — reap releases ownership. Once a connection has left `registered` it owns nothing.

I3 — at most one live owner per identity. A live connection is never blocked from claiming a scene by a dead predecessor of the same declared identity that still, on paper, owns it.

I4 — reconnect inherits nothing special (settled). Unchanged from round 1; `Connect`'s definition has not changed.

Two are new as of the multi-path refinement, and remain the reason that round exists:

I5 — leaving the registry iff released, however triggered. The general form of I2: no reachable state has a connection absent from `registered` while it still owns a scene, *regardless of which operation removed it*, and no read operation ever removes anyone. This is what closes H1 and H2 – H1 broke it via a read removing without releasing, H2 broke it via a graceful departure removing without releasing; I5 rules out both shapes at once, not just the lease-lapse shape I2 alone would cover.

I6 — a live writer keeps its ownership. A connection that keeps making authenticated contact by writing is never reaped, by its own write or another's, while its transport stays up. This is what closes H3: it is I5's negation specialized to the case where no transport ever died, so the only way to reach it would be a write path that swept its own caller.

2 Basic Types

$[CONN, IDENT, SCENE]$

Unchanged from round 1. Three `CONN` values suffice to model an old and a new connection sharing one identity, plus one connection of a second identity, plus (for the H3 trace) a distinct claimant. Two `IDENT` values exhibit the same-identity and cross-identity cases side by side. Two `SCENE` values show one scene surviving a departure intact beside one that does not. All three carriers are bounded by ProB's `DEFAULT_SETSIZE`.

3 Free Types

$TSTATE ::= tup \mid tdown$

Whether a connection's transport is still alive, unchanged from round 1.

$LSTATE ::= llive \mid llapsed$

A connection's lease, abstracted to whether it has lapsed, unchanged from round 1 in its two values – but see `LeaseLapse` below for what changed in when it may transition.

$OWNER ::= unowned \mid conn\langle\langle CONN \rangle\rangle$

Unchanged from round 1.

4 State

Unchanged from round 1: one flat schema, structural coherence only, the safety properties deliberately absent so a violating operation is a counterexample rather than a disabled transition.

ConnReg

$$
\begin{array}{l}
registered : \mathbb{P}\ CONN \\
identity : CONN \pfun IDENT \\
transport : CONN \pfun TSTATE \\
lease : CONN \pfun LSTATE \\
owner : SCENE \fun OWNER \\
\hline
registered \subseteq \mathrm{dom}\ identity \\
\mathrm{dom}\ identity = \mathrm{dom}\ transport \\
\mathrm{dom}\ transport = \mathrm{dom}\ lease
\end{array}
$$

As in round 1, `identity` is kept for every connection that has ever connected, including a departed one – I3's same-identity comparison needs it, and no operation below ever shrinks `dom identity`. `owner` stays total over `SCENE` for the same reason round 1 gave: every predicate below is then a plain function application, never a domain-membership guard.

5 Initialization

Init

$$
\begin{array}{l}
ConnReg' \\
\hline
registered' = \emptyset \\
identity' = \emptyset \\
transport' = \emptyset \\
lease' = \emptyset \\
owner' = (\lambda\, s : SCENE \bullet unowned)
\end{array}
$$

Unchanged from round 1.

6 Operations

6.1 Connect — arrival and reconnect, undifferentiated (I4)

Unchanged from round 1: no precondition, so nothing here privileges a fresh connection over a reconnecting one, and nothing here evicts or transfers another connection's state. This absence of a guard remains the whole of I4's argument.

Connect

$\Delta ConnReg$
$c? : CONN$
$i? : IDENT$

$registered' = registered \cup \{c?\}$
$identity' = identity \oplus \{c? \mapsto i?\}$
$transport' = transport \oplus \{c? \mapsto tup\}$
$lease' = lease \oplus \{c? \mapsto llive\}$
$owner' = owner$

6.2 Renew — authenticated contact that is not a write

Models `HubClientRegistry.record` called without an `AddElement/SetProperty/RemoveElement` attached to it – an explicit `identify` or `register_client` call. Guarded on `transport c? = tup`, unchanged from round 1: a connection with a dead transport cannot make contact of any kind, so it cannot renew this way either.

Renew

$\Delta ConnReg$
$c? : CONN$

$c? \in registered$
$transport\ c? = tup$
$lease' = lease \oplus \{c? \mapsto llive\}$
$registered' = registered$
$identity' = identity$
$transport' = transport$
$owner' = owner$

6.3 TransportDies — the environment, not the Hub

Unchanged from round 1.

TransportDies

$\Delta ConnReg$
$c? : CONN$

$c? \in registered$
$transport\ c? = tup$
$transport' = transport \oplus \{c? \mapsto tdown\}$
$registered' = registered$
$identity' = identity$
$lease' = lease$
$owner' = owner$

6.4 LeaseLapse — time passes with nothing to renew it

This is the one carry-over operation that changes. Round 1 guarded `LeaseLapse` on `transport c? = tdown`, reasoning that a live transport implies contact keeps renewing the lease. H3 is the proof that reasoning was wrong: `apply` is contact over a live transport that, in the shipped code, renews nothing. The guard is therefore weakened to drop the transport condition entirely – a lease lapses whenever its length has passed since the last renewal, live transport or not, exactly matching `SessionLease.is_live`, which is pure time arithmetic and never inspects transport state.

LeaseLapse

$\Delta ConnReg$
$c? : CONN$

$c? \in registered$
$lease\ c? = llive$
$lease' = lease \oplus \{c? \mapsto llapsed\}$
$registered' = registered$
$identity' = identity$
$transport' = transport$
$owner' = owner$

6.5 Read — the non-destructive query

Models `HubClientRegistry.named_sessions`, `live_sessions`, and `repos` *as this fix requires them to behave*: a read changes nothing. Ξ `ConnReg` is the whole schema – there is no parameter and no predicate beyond the implicit `registered' = registered` (and every other component held) that Ξ carries. This is I5's second half stated as a structural fact about `Read`'s own definition, the same way I4 was argued structurally in round 1: no reachability check is needed to show a schema with no successor-changing predicate cannot change `registered`. Section 9.1's control is what `Read` looks like without this fix.

Read

$\Xi ConnReg$

6.6 Depart — the single removal-and-release coordinator

Models `HubDisplay.drop_connection`, fixed to do what its own docstring today says it deliberately does not: release. One schema stands for both triggers the ruling names as "graceful" and "idle-reap," because both concretely call the identical `drop_connection` today, via `session_cleanup.py`'s teardown leg and `lifecycle.disconnect_connection` – there is exactly one code path, so there is exactly one schema. The guard is only $c? \in registered$: no lease condition, because a session ending cleanly or being idle-reaped by the SDK happens regardless of whether the Hub's own lease has separately lapsed it. Removal and release are one predicate, one step: there is no state in which `c?` has left `registered` under this operation without `owner` already reflecting the release.

Depart

$\Delta ConnReg$
$c? : CONN$

$c? \in registered$
$registered' = registered \setminus \{c?\}$
$owner' = owner \oplus$
$\quad (\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
$identity' = identity$
$transport' = transport$
$lease' = lease$

6.7 TimedReap — the lease as a real background backstop

`Depart` and `Claim`'s embedded sweep are both *pull-based*: each fires only because some party – a disconnecting client, the SDK's idle-reap, or a peer's write – happened to act. Neither fires on its own. The bead this model exists to close reports a connection that persisted 25.79 hours: no peer ever wrote again, and nothing ever disconnected it, so neither pull-based path ever ran, and a purely pull-based fix would have reproduced exactly that silence. `TimedReap` is the same atomic deregister-and-release predicate as `Depart`, but with `Depart`'s permissive guard narrowed to the one condition a background timer can act on without any other party's help: the lease has lapsed. No other connection needs to exist, act, or have ever existed, for `TimedReap(c?)` to become and stay enabled.

TimedReap

$\Delta ConnReg$
$c? : CONN$

$c? \in registered$
$lease\ c? = llapsed$
$registered' = registered \setminus \{c?\}$
$owner' = owner \oplus$
$\quad (\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
$identity' = identity$
$transport' = transport$
$lease' = lease$

The predicate is line-for-line `Depart`'s, with one added conjunct in the guard. That repetition is deliberate, not an oversight: Section 7's two-lock argument depends on every registry-mutating operation sharing exactly this shape, and stating `TimedReap` as "`Depart` restricted to a lapsed lease" in prose, without restating its predicate in full, would leave a reader unable to check by inspection that the restriction is the *only* difference. `TimedReap` is, in fact, round 1's original `Reap` operation, reinstated verbatim: round 1 was right that a lapsed lease alone should be enough to trigger release, and this addendum's ruling is that round 1 was also right to make it a freestanding, unconditional operation rather than something this round's first cut folded away entirely into `Depart` and `Claim`. The difference from round 1 is only what it now stands *beside*: an explicit non-destructive `Read`, an explicit external-signal `Depart`, and a `Claim` that also renews and sweeps – so the model no longer has to choose between "the lease fires on its own" and "leaving the registry always releases ownership, however triggered"; it has both.

6.8 Claim — the ownership-gated write, renewing itself first

Models `HubDisplay.apply`: `SetProperty`/`RemoveElement`'s `OwnerTracker.require_ownership` check, and (folded into the same guard, as in round 1) `AddElement`'s grant onto a not-yet-owned scene. **Three things happen in one atomic step, in this order, exactly matching `apply`'s own docstring:**

1. **Renew the caller.** $lease' = lease \oplus \{c? \mapsto llive\}$ happens unconditionally, before anything else is computed. This is the ruling's third decision – an authenticated write is contact, and contact renews – and it is the one line the shipped code is missing (`apply` never calls `HubClientRegistry.record` for its own caller).

2. **Sweep every *other* lapsed connection.** The set $\{c2 : CONN \mid c2 \in registered \land c2 \neq c? \land lease\ c2 = llapsed\}$ names every connection but the caller whose lease has lapsed; each is dropped from `registered` and every scene it owned is released to `unowned`, in the same step as the renewal above. The $c2 \neq c?$ exclusion is what step 1 buys: because `c?` was just renewed to `llive`, it could never satisfy this set's own guard anyway, but stating the exclusion explicitly is what distinguishes this schema from Section 9.3's control, where the exclusion is exactly what is missing.

3. **Claim, against the swept result.** The ownership guard – `unowned` or already the caller's – is evaluated against `owner` *after* step 2's release, matching `apply`'s docstring: "reaps and releases first, before the ownership check." A reconnecting client claiming a scene a departed, same-call-swept predecessor held is not shadowed by it.

`transport c? = tup` is required as a precondition, matching `Renew`: an authenticated write, like any authenticated contact, can only arrive over a live transport.

Claim

$$
\begin{array}{l}
\Delta ConnReg \\
c? : CONN \\
s? : SCENE \\
\hline
c? \in registered \\
transport\ c? = tup \\
lease' = lease \oplus \{c? \mapsto llive\} \\
registered' = registered \setminus \\
\quad \{c2 : CONN \mid c2 \in registered \land c2 \neq c? \land lease\ c2 = llapsed\} \\
(owner \oplus (\{s : SCENE \mid \exists\, c2 : CONN \bullet \\
\quad c2 \in registered \land c2 \neq c? \land lease\ c2 = llapsed \land owner\ s = conn(c2)\} \\
\quad \times \{unowned\}))\ s? = unowned \\
\lor \\
(owner \oplus (\{s : SCENE \mid \exists\, c2 : CONN \bullet \\
\quad c2 \in registered \land c2 \neq c? \land lease\ c2 = llapsed \land owner\ s = conn(c2)\} \\
\quad \times \{unowned\}))\ s? = conn(c?) \\
owner' = (owner \oplus (\{s : SCENE \mid \exists\, c2 : CONN \bullet \\
\quad c2 \in registered \land c2 \neq c? \land lease\ c2 = llapsed \land owner\ s = conn(c2)\} \\
\quad \times \{unowned\})) \oplus \{s? \mapsto conn(c?)\} \\
identity' = identity \\
transport' = transport
\end{array}
$$

7 Invariants

None of the six properties is placed in the `ConnReg` schema, for the same reason round 1 gave: a predicate there would be enforced as a guard on every successor, silently disabling the very operation that would break it.

I1 — transport-gone implies *unconditionally* eventually reaped. As in round 1, this is a liveness property argued over enabledness and reachability, not a single-state safety formula – Z has no temporal operator of its own, so the claim is: for any `c?` with `transport c? = tdown`, no reachable state traps `c?` in `registered` forever, and once it leaves, every scene it owned is already released. `Renew`'s guard means a connection can never re-arm its lease once its transport has died without an intervening `Connect`. Once $transport\ c? = tdown \land lease\ c? = llive$, `LeaseLapse` is enabled and depends on nothing but `c?`'s own state; once `lease c? = llapsed`, `TimedReap` (Section 6.7) is enabled and *also* depends on nothing but `c?`'s own state – not on any other connection existing, not on any read ever running, not on any disconnect ever being detected. This is the strengthening this addendum makes, and it is worth being exact about what was actually wrong with the previous round's argument, because it is not a reachability gap. `Depart`'s guard, $c? \in registered$, was *already* satisfiable with no second connection and no `Read` in the picture – a `fuzz` reachability check alone cannot distinguish "this operation's formal precondition holds" from "something in the real system will actually invoke it," because Z enabledness is silent on triggers. The error in trusting that chain as *unconditional* was informal, not formal: this document's own prose already restricted `Depart` to modelling a real disconnect signal or the SDK's idle-reap (Section 6.6), and for a killed process neither ever arrives – so `Depart`'s formal availability was never in question, only whether anything in the real system would ever exercise it. `TimedReap` closes that informal gap by being an operation whose intended trigger – time passing – genuinely requires nothing external, which is a claim about what invokes it in the real `HubClientRegistry`, not one `fuzz` or `probcli` can check for either operation. What *is* newly, formally checkable is that this reading is now backed by an operation whose guard depends on nothing but `c?`'s own recorded state, so a periodic caller with no reference to any other connection's identity, and no external signal delivered to it at all, could implement exactly this guard: the chain `TransportDies ; LeaseLapse ; TimedReap`, run against a carrier holding only `c?` itself – no second connection, no `Read`, no `Depart` – is a complete, self-contained witness that `c?` leaves `registered` with every scene it owned released, and Section 8 confirms this trace by reachability with `CONN` instantiated to a single element. `Depart` and `Claim`'s embedded sweep remain additional, faster dischargers of a lapsed lease when their own triggers happen to fire first (Section 7 shows the three do not race); they are no longer I1's *only* route, which is what round 1's original coverage

audit named as an open gap and this addendum closes.

I2 — reap releases ownership.

$$\neg\ \exists\ s : SCENE;\ c : CONN \bullet owner\ s = conn(c) \land c \notin registered.$$

Restated verbatim from round 1. It remains true, but it is no longer the whole of the argument, because there are now three operations that can remove a connection from `registered` (`Depart`, `TimedReap`, and `Claim`'s embedded sweep of others) instead of one, plus one that must never remove anyone (`Read`). I5, below, is the version of this argument that actually accounts for all four.

I3 — at most one live owner per identity.

$$\begin{aligned}\neg\ \exists\ & s : SCENE;\ c, c' : CONN \bullet \\ & c \in registered \land c' \notin registered \land \\ & c' \in \mathrm{dom}\ identity \land identity\ c = identity\ c' \land \\ & c \neq c' \land owner\ s = conn(c').\end{aligned}$$

Unchanged in statement from round 1: I5's general form already rules out *any* scene being owned by a connection outside `registered`, so I3 holds as the same-identity corollary of I5 that round 1 argued it was of I2.

I4 — reconnect inherits nothing special (settled). Unchanged from round 1 – `Connect`'s definition has not changed, and the argument is unchanged: no successor of `Connect` touches any other connection's state, so a reconnecting client's only route to ownership is the same `Claim` every connection uses.

I5 — leaving the registry iff released, however triggered.

$$\neg\ \exists\ s : SCENE;\ c : CONN \bullet owner\ s = conn(c) \land c \notin registered$$

together with: no operation whose only effect is a read (that is, `Read` itself) ever changes `registered`.

The first conjunct is I2's formula, now proven against four removal paths instead of one: `Depart` and `TimedReap` each release every scene `c?` owned in the same step they remove `c?` (Sections 6.6 and 6.7 – the two predicates are identical apart from `TimedReap`'s added lease guard, so this argument covers both at once), and `Claim`'s embedded sweep releases every scene each swept `c2` owned in the same step it removes `c2` from `registered` (Section 6.8's step 2). No other operation ever assigns `owner` to a connection outside `registered` – `Claim`'s own guard, $c? \in registered$, and its postcondition, $c? \in registered'$ always (Section 6.8's $c2 \neq c?$ exclusion guarantees the caller is never among the connections it removes), rule that out at the point of assignment. The second conjunct is `Read`'s own definition (Section 6.5): Ξ `ConnReg` carries `registered' = registered` by construction, so this half needs no reachability check, exactly as I4 needed none.

This is what closes H1 and H2 together, as one invariant rather than two patches: H1 broke the second conjunct (a read removed someone), H2 broke the first conjunct via a specific operation (`Depart` removing without releasing); I5 states the shape both holes share – *leaving the registry and releasing ownership are the same event, no matter which of the model's removing operations produced it, and a non-removing operation produces neither* – so that a fourth departure trigger appearing later in the code would have to be checked against the same general property, not against a fourth ad hoc patch. `TimedReap` *is* that fourth trigger, added within this same round, and I5's statement did not need to change at all to absorb it – only its proof gained one more, textually near-identical, case.

I6 — a live writer keeps its ownership.

$$\neg\ \exists\ s : SCENE;\ c : CONN \bullet owner\ s = conn(c) \land c \notin registered \land transport\ c = tup.$$

I5's negation, specialized to the case where the departing connection's transport never died. Every witness to I5's general formula that `Depart` could produce requires nothing of transport state at all (Section 6.6's guard is orthogonal to it), so it does not distinguish this case on its own – but that is exactly why `c?`'s own `transport` being `tup` throughout a trace is a meaningful discriminator elsewhere: a `TimedReap`-produced

witness *also* requires nothing of transport state, so a witness confined to `transport c? = tup` could still, in principle, come from either `Depart` or `TimedReap` acting on a connection that never crashed at all – an ordinary graceful departure or a lease that genuinely lapsed from real inactivity, neither of which is a bug. What I6 rules out is narrower and sharper: that `Claim`'s own sweep, not `Depart` or `TimedReap`, ever catches its *own caller* on the very call that caller is making – the one mechanism H3 names, and the one no legitimate departure operation is even attempting to perform mid-write. In the fixed spec that cannot happen, because `Claim`'s $c2 \neq c?$ exclusion (Section 6.8, step 2) means the caller can never appear in the set it sweeps, regardless of what its own `lease` showed on entry. Section 9.3's control removes exactly that exclusion and shows the formula above becomes reachable in three steps that never call `TransportDies`, `Depart`, or `TimedReap` at all – isolating the claim to `Claim` alone, exactly as the fidelity-control discipline requires.

The two-lock concern is moot by construction. The current, unfixed code holds `HubClientRegistry`'s own lock around registration and `OwnerTracker` under no lock of its own (`HubDisplay`'s store lock covers it instead) – two different locks, held by two different objects, and H1/H2 are exactly what happens when a caller updates one without the other under one critical section. This model does not need a second lock in its own terms, because it does not need coordinating in the first place: `Read` never mutates `registered` (Section 6.5), so the only three operations that ever do – `Depart`, `TimedReap`, and `Claim` – are also, by their own single predicate, the only three operations that ever mutate `owner` on a departure. There is no interleaving in which one without the other could be observed, because in this model, as the fix requires of the real `HubDisplay` (the one object holding both `_clients` and `_owners`), removing-from-the-registry and releasing-ownership are not two updates that a lock discipline keeps in step; they are one update.

The three coordinators serialize; none of them races. Round 2's implementation-facing question is whether a background timer firing `TimedReap` can race `Depart` (an external disconnect arriving at the same moment) or `Claim` (a write arriving at the same moment) on the *same* connection, and land the registry or the ownership table in a state neither operation intended. In this model the question has a structural answer, not merely an empirical one: a Z operation schema denotes a single, indivisible relation between a before-state and an after-state, and `DEFAULT_SETSIZE`-bounded model-checking with ProB explores every possible *interleaving* of enabled operations, never a partial or torn application of one. There is no notion in this model of two operations executing "at once" and observing each other mid-update – only of one full step happening, then the next full step choosing among whatever is enabled in the resulting state. Three consequences follow, and Section 8's full `-model_check` (which visits every reachable state and reports no deadlock) is what confirms none of them is merely assumed:

1. **Whichever fires first wins, and correctly.** If `TimedReap(c?)` and `Depart(c?)` are *both* enabled in some state (a lapsed lease and a live, graceful-disconnect-eligible connection are not mutually exclusive conditions), firing either one removes `c?` from `registered` and releases every scene it owned – the two predicates differ only in `TimedReap`'s extra guard, never in their effect. The second operation, whichever it is, is then simply disabled: its guard, $c? \in registered$, fails, because the first one already removed `c?`. There is no state in which both fire on the same `c?` and no state in which their effects could conflict, because they compute the identical postcondition.

2. **A write is never stranded behind a timer, or vice versa.** `Claim(c2, s?)` for a *different* connection `c2` and `TimedReap(c1)` for a lapsed `c1` are independent: neither's guard reads the other's parameter, and `Claim`'s own embedded sweep (Section 6.8, step 2) would perform the identical release on `c1` if `TimedReap` had not already done so first. Whichever fires first, the reachable state after both have run (in either order) is the same: `c1` departed and released, `c2`'s claim resolved against a registry that no longer shows `c1` as a candidate to be swept. Reachability is monotonic under adding `TimedReap` to a spec that already had `Depart` and `Claim`: every state and trace reachable before this addendum remains reachable after it, because no existing operation's guard or effect changed.

3. **A connection can never claim while being timed-reaped out from under itself.** `Claim(c?, s?)` renews `c?`'s own lease to `llive` as its first effect (Section 6.8, step 1), and `TimedReap(c?)` requires `lease c? = llapsed`. Because both are whole, indivisible steps, there is no state in which `Claim(c?, s?)` is "in progress" for `TimedReap(c?)` to observe a stale lease during: either

`Claim` has already completed, in which case `c?`'s lease is `llive` and `TimedReap(c?)` is disabled, or it has not yet run at all.

None of this needs a fourth invariant: it is I5 and I6, reread as claims about *any* pair of the four registry-mutating operations rather than about `Claim` alone, and both are already checked exhaustively over every reachable state, from every reachable ordering of the four, by Section 8's I5/I6 goal checks. A genuine race producing an inconsistent intermediate state would surface there directly, as a `FOUND` counterexample naming the offending trace – not merely as a deadlock, which Section 8's separate full model-check also confirms absent, so neither failure mode is left unchecked.

8 Verification

Type-checking is a `fuzz` pass:

```
fuzz docs/connection_lease_reaping.tex
```

I2, I3, I5, and I6 are safety properties, checked by asking ProB whether their negation is *reachable*. A goal reported *not found* means the invariant holds on every reachable state.

```
# I2 (goal NOT found in this, the fixed, spec)
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered)))" \
  -p DEFAULT_SETSIZE 3

# I3 (goal NOT found in this, the fixed, spec)
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#s.(#c.(#c2.(s:SCENE & c:CONN & c2:CONN &
                      c : registered & not(c2 : registered) &
                      c2 : dom(identity) & identity(c) = identity(c2) &
                      not(c = c2) & owner(s) = conn(c2))))" \
  -p DEFAULT_SETSIZE 3

# I5 (identical goal to I2 -- I5's first conjunct; goal NOT found)
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered)))" \
  -p DEFAULT_SETSIZE 3

# I6 (goal NOT found in this, the fixed, spec)
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered) & transport(c) = tup))" \
  -p DEFAULT_SETSIZE 3
```

I1's mechanism is confirmed by reachability of its positive outcome:

```
# I1's chain completes (goal FOUND): a connection that connected, died,
# and departed is outside registered with a dead transport
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
             transport(c) = tdown)" \
  -p DEFAULT_SETSIZE 3
```

The state a witness ends in does not, by itself, say which operation produced it: `Depart` and `TimedReap` compute the identical postcondition (Section 6.7), so a reachability goal alone cannot certify that `TimedReap` specifically was exercised. The same goal remains *FOUND* at `DEFAULT_SETSIZE 1`, which bounds `CONN` to a single element and so rules out any second connection appearing anywhere in the witness by construction of the carrier:

```
# goal FOUND even with only one connection in the whole carrier
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
             transport(c) = tdown)" \
  -p DEFAULT_SETSIZE 1
```

The printed witness is not reproducible evidence on its own – ProB's search order is not obliged to return the same trace on every run, and a run of this exact check returned a 4-step `Depart`-based trace on one invocation and a 6-step one passing through `Claim` and `LeaseLapse` on another, because `Depart`'s guard, $c? \in$ *registered* alone, was already satisfiable the moment `TransportDies` fires and does not require the lease to have lapsed at all. What is reproducible, and what actually proves the strengthening, is the structural argument already given above: at any point where `lease c? = llapsed` holds for a registered `c?` – which `LeaseLapse`'s postcondition establishes unconditionally, independent of any other connection – `TimedReap(c?)` is enabled by inspection of its own guard, whether or not the same state also happens to enable `Depart`. A search tool that reports *some* witness cannot be asked to prefer one enabled operation over another that produces the identical effect; the schema's own precondition is what the proof rests on, not which branch a particular run's search order happened to take.

The fix's positive outcome – a live connection successfully claims a scene its departed, same-identity predecessor used to own – is also confirmed reachable:

```
# A live connection owns a scene a departed same-identity predecessor
# used to own (goal FOUND).
probcli docs/connection_lease_reaping.tex -model_check -nodead \
  -goal "#s.(#c.(#c2.(s:SCENE & c:CONN & c2:CONN & c : dom(identity) &
                      not(c = c2) & identity(c) = identity(c2) &
                      not(c : registered) & c2 : registered &
                      owner(s) = conn(c2))))" \
  -p DEFAULT_SETSIZE 3
```

Deadlock. `Connect` carries no precondition beyond its two arguments' types, so it is enabled in every reachable state; the full model-check confirms no deadlock over the bounded carrier:

```
probcli docs/connection_lease_reaping.tex -model_check -p DEFAULT_SETSIZE 3
```

9 Fidelity: reproducing the three shipped holes

A model that cannot reproduce a known defect proves nothing about closing it. Each hole gets its own control, changing exactly one thing from this, the fixed, spec – so that a goal found in one control and not found in the fixed spec attributes the defect to that one change and no other.

9.1 H1 — destructive-read-sweep

`docs/connection_lease_reaping_destructive_read_buggy.tex` changes only `Read`: instead of Ξ `ConnReg`, it sweeps a lapsed connection out of `registered` exactly as `HubClientRegistry._sweep_locked` does today, and leaves `owner` untouched – `named_sessions` discards the swept set's identity the moment it returns, so nothing downstream can release what it just removed.

```
% fixed (Read, this spec):
\Xi ConnReg

% buggy (Read, the H1 control): the same removal Depart performs, but
% with no release, fired from a query instead of the coordinator
registered' = registered \ {c? | c? : registered & lease(c?) = llapsed}
owner' = owner
```

The trace.

1. `Connect`(c1, i1): `registered` = {c1}.

2. `Claim`(c1, s1): `owner(s1) = conn(c1)`.

3. `TransportDies`(c1): `transport(c1) = tdown`.

4. `LeaseLapse`(c1): `lease(c1) = llapsed`.

5. `Read` – buggy: `registered = {}` (c1 swept), but `owner(s1) = conn(c1)` is left exactly as it was. In the fixed spec, `Read` is Ξ `ConnReg` and this step does nothing at all – `registered` still holds `c1`.

6. `Connect`(c2, i1): a live reconnect under the same identity.

7. `Claim`(c2, s1): under the buggy variant, `owner(s1) = conn(c1)` and $c1 \neq c2$, so `Claim` is disabled – the reconnecting client is shadowed by a connection a read, not even a write, silently removed. Worse than round 1's shape: no operation in this variant can ever release `s1`, because c1 is no longer `registered` for `Depart` or `Claim`'s sweep to find. In the fixed spec, step 5 changed nothing, so `c1` is still `registered` with a lapsed lease; a subsequent `Depart`(c1) or `Claim` by any third connection releases `s1` normally, and `Claim`(c2, s1) succeeds once it does.

```
# must be FOUND against the H1 control, NOT found against the fixed spec
probcli <spec>.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered)))" \
  -p DEFAULT_SETSIZE 3
```

9.2 H2 — graceful-departure-without-release

`docs/connection_lease_reaping_no_release_buggy.tex` changes only `Depart`: it still removes `c?` from `registered`, but `owner` is left untouched – the literal current behaviour of `HubDisplay.drop_connection`, whose own docstring says the departing connection's roots "stay owned by its id." Round 1 kept this filename for a variant of the *lease-lapse* path; this round retargets the same filename to the operation the review actually found broken, `Depart`, and the trace below never lapses a lease at all – `c1`'s transport never dies, matching the review finding that this is the ordinary, live-session departure path, not a timeout path.

```
% fixed (Depart, this spec):
registered' = registered \ {c?}
owner' = owner (+) ({s : SCENE | owner(s) = conn(c?)} * {unowned})

% buggy (Depart, the H2 control): the literal shipped drop_connection
registered' = registered \ {c?}
owner' = owner
```

The trace.

1. `Connect`(c1, i1): `registered = {c1}`, `transport(c1) = tup` throughout – this connection's transport never dies.

2. `Claim`(c1, s1): `owner(s1) = conn(c1)`.

3. `Depart`(c1) – buggy, an ordinary graceful disconnect, no lease condition required: `registered = {}`, but `owner(s1) = conn(c1)` unchanged.

4. `Connect`(c2, i1); `Claim`(c2, s1) is disabled – the reconnecting client is shadowed by a connection that left cleanly, with a live transport, not a crashed one.

```
# must be FOUND against the H2 control, NOT found against the fixed spec
probcli <spec>.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered)))" \
  -p DEFAULT_SETSIZE 3
```

9.3 H3 — self-reap-without-renew

`docs/connection_lease_reaping_self_reap_buggy.tex` changes only `Claim`: it omits the self-renewal step and the $c2 \neq c?$ exclusion from the sweep it performs, so the caller is swept by its own, ordinary sweep exactly as `HubDisplay._reap_and_release` sweeps every session including its own caller's, because `apply` never calls anything that renews the caller first.

```
% fixed (Claim, this spec): renew self first, sweep only OTHERS
lease' = lease (+) {c? |-> llive}
registered' = registered \ {c2 : CONN | c2 : registered & c2 /= c? &
                                        lease(c2) = llapsed}

% buggy (Claim, the H3 control): no self-renewal, no self-exclusion --
% the caller is swept exactly like anyone else
lease' = lease
registered' = registered \ {c2 : CONN | c2 : registered &
                                        lease(c2) = llapsed}
```

The trace. `transport(c1) = tup` at every step – this connection never disconnects, never crashes, and never has its transport die. It is reaped purely because its own writes never renew it.

1. `Connect`(c1, i1): `registered` = {`c1`}, `lease(c1) = llive`.
2. `Claim`(c1, s1) – the initial show: guard passes (`owner(s1) = unowned`); `owner(s1) = conn(c1)`. Under the buggy variant, `lease(c1)` is left exactly as it was – still `llive` at this point, so no divergence from the fixed spec is visible yet.
3. `LeaseLapse`(c1): enabled because `lease(c1) = llive` and (Section 6.4) this operation no longer requires a dead transport – `lease(c1) = llapsed`. Nothing renewed `c1` between steps 2 and 3; this is the patching connection ageing past its own TTL with only writes, no explicit `identify`, in between.
4. `Claim`(c1, s1) again – a later patch, buggy: the sweep set is $\{c2 \mid c2 \in \mathit{registered} \wedge \mathit{lease}\ c2 = \mathit{llapsed}\} = \{c1\}$, since nothing excludes the caller. $\mathit{registered}' = \mathit{registered} \setminus \{c1\} = \varnothing$: *c1 removes itself.* `owner` is first released for every scene c1 owned ($\mathit{owner}(s1) \mapsto \mathit{unowned}$), then the claim in this same call re-grants `s1` to `conn(c1)` – so `owner(s1) = conn(c1)` again, but $c1 \notin \mathit{registered}$: I5/I6's negation is satisfied by this single step. Under the fixed spec, the same step's sweep set excludes `c1` by construction, so `registered' =` {`c1`} – c1 stays registered, and the renewal in step 1 of `Claim` means this exact sequence could not have reached `lease(c1) = llapsed` again without a fresh, unrenewed gap.

ProB's search confirms an even shorter witness than the hand-derived one above: `Connect`(c1, i1); `LeaseLapse`(c1); `Claim`(c1, s1) – three steps, no successful claim needed before the lapse at all, because `LeaseLapse` only needs `lease(c1) = llive`, which `Connect` already establishes. The four-step version above is kept in prose because it names the scenario the review actually reported – an initial show followed by a later patch – but the mechanism is identical: whichever step first exposes a lapsed lease at the caller's own next `Claim`, the buggy sweep catches it.

```
# must be FOUND against the H3 control, NOT found against the fixed spec
probcli <spec>.tex -model_check -nodead \
  -goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
                 not(c : registered) & transport(c) = tup))" \
  -p DEFAULT_SETSIZE 3
```